Stack Operations Using an Array (C Programming)

Name: _____ Roll No.: _____ GitHub: _____

1. Program Analysis

Item	Details
Data structure	Stack (LIFO), implemented with a fixed-size array. Insertion and removal happen only at the top.
Variables	Global: stack[SIZE] (int array), top (int). Local: item in main() and in pop(); i in display().
Constant	SIZE = 5 (macro), so valid indices are 0..4.
Initial values	top = -1 (empty stack); stack[] elements are 0 because globals are zero-initialised, but they are not part of the stack until pushed.
Functions	push(int item) - void; pop() - returns int (-1 on underflow); display() - void; main().
Conditions	push: top == SIZE-1 (full, Overflow). pop: top == -1 (empty, Underflow). display: top == -1 (print empty message).
Loops	One loop: for (i = 0; i <= top; i++) in display(), prints bottom to top. No loops in push/pop (O(1)).
Key invariant	-1 <= top <= SIZE-1; number of elements = top + 1.

2. Manual Trace Table (done before execution)

Stack shown bottom to top. 'top' is the value after the operation. Entries in the array above top are stale and ignored.

#	Operation	item	top	Stack	Output
0	Start	-	-1	[ ]	-
1	push(10)	10	0	[10]	Pushed: 10
2	push(20)	20	1	[10,20]	Pushed: 20
3	push(30)	30	2	[10,20,30]	Pushed: 30
4	display()	-	2	[10,20,30]	Stack: 10 20 30
5	pop()	30	1	[10,20]	(none, returns 30)
6	printf	30	1	[10,20]	Popped: 30
7	push(40)	40	2	[10,20,40]	Pushed: 40
8	push(50)	50	3	[10,20,40,50]	Pushed: 50
9	push(60)	60	4	[10,20,40,50,60]	Pushed: 60
10	push(70)	70	4	[10,20,40,50,60]	Stack Overflow (top==4==SIZE-1, no change)
11	display()	-	4	[10,20,40,50,60]	Stack: 10 20 40 50 60
12	pop() + printf	60	3	[10,20,40,50]	Popped: 60
13	pop() + printf	50	2	[10,20,40]	Popped: 50
14	pop() + printf	40	1	[10,20]	Popped: 40
15	pop() + printf	20	0	[10]	Popped: 20
16	pop() + printf	10	-1	[ ]	Popped: 10
17	display()	-	-1	[ ]	Stack is Empty
18	pop() (top==-1)	-1	-1	[ ]	Stack Underflow
19	printf	-1	-1	[ ]	Popped: -1

3. Function Calls, Return Values and Loop Variable i

Call	Returns	Notes
push(10), push(20), push(30)	void	each: top++, store, print
display() #1	void	top=2: i = 0, 1, 2 (prints 10, 20, 30); i=3 fails i<=top, loop ends
pop() #1	30	top 2 -> 1
push(40), push(50), push(60)	void	top 1 -> 2 -> 3 -> 4
push(70)	void	overflow branch, top stays 4
display() #2	void	top=4: i = 0,1,2,3,4 (prints 10,20,40,50,60); ends at i=5
pop() #2 - #6	60, 50, 40, 20, 10	top 4 -> 3 -> 2 -> 1 -> 0 -> -1
display() #3	void	top=-1: loop NOT executed (i never assigned); prints empty message
pop() #7	-1	underflow branch; top unchanged

4. Predicted Output and Final Stack State

Predicted before execution (trailing space after the last number in each 'Stack:' line comes from printf("%d ")):

```
Pushed: 10
Pushed: 20
Pushed: 30
Stack: 10 20 30
Popped: 30
Pushed: 40
Pushed: 50
Pushed: 60
Stack Overflow
Stack: 10 20 40 50 60
Popped: 60
Popped: 50
Popped: 40
Popped: 20
Popped: 10
Stack is Empty
Stack Underflow
Popped: -1
```

Final state: top = -1, the stack is empty (logically []). Physically stack[0..4] still holds 10, 20, 40, 50, 60 as stale data, which is never read because every access is guarded by top.

5. Overflow and Underflow

Stack Overflow: caused by `push(70)`. After push(60) the stack holds 5 items, so top = 4 = SIZE-1 and the condition top == SIZE - 1 is true.

Stack Underflow: caused by the **last** `pop()` in main(), called after the stack was already emptied (top = -1). It returns -1, and main() prints 'Popped: -1'. This -1 is an error marker, not a real data item.

6. Modified Program and Trace

Modification: (a) input values changed: push(10), push(20), push(30) became push(15), push(25), push(35); (b) operation changed: push(70) replaced by `item = pop(); printf("Popped: %d\n", item);`. Source: modified.c.

#	Operation	item	top	Stack	Output
1-3	push 15, 25, 35	-	2	[15,25,35]	Pushed: 15 / 25 / 35
4	display()	-	2	[15,25,35]	Stack: 15 25 35
5	pop()	35	1	[15,25]	Popped: 35
6-8	push 40, 50, 60	-	4	[15,25,40,50,60]	Pushed: 40 / 50 / 60

#	Operation	item	top	Stack	Output
9	pop() (replaces push(70))	60	3	[15,25,40,50]	Popped: 60 (no overflow now)
10	display()	-	3	[15,25,40,50]	Stack: 15 25 40 50
11-14	pop() x4	50,40,25,15	-1	[ ]	Popped: 50 / 40 / 25 / 15
15	pop() (5th)	-1	-1	[ ]	Stack Underflow, Popped: -1
16	display()	-	-1	[ ]	Stack is Empty
17	pop()	-1	-1	[ ]	Stack Underflow, Popped: -1

Predicted output of modified program:

```
Pushed: 15
Pushed: 25
Pushed: 35
Stack: 15 25 35
Popped: 35
Pushed: 40
Pushed: 50
Pushed: 60
Popped: 60
Stack: 15 25 40 50
Popped: 50
Popped: 40
Popped: 25
Popped: 15
Stack Underflow
Popped: -1
Stack is Empty
Stack Underflow
Popped: -1
```

Effect: the overflow disappears but two underflows now occur, because the program still performs 5 pops after only 4 items remain. **Final state:** empty, top = -1.

7. Logical Errors and Limitations

#	Issue	Why it matters	Correction
1	pop() returns -1 on underflow	-1 is also a legal data value. push(-1) then pop() gives -1, indistinguishable from an error (test T3).	Return a status code and pass the value through a pointer: int pop(int *item). See stack_fixed.c.
2	push() is void	Caller cannot tell whether the push succeeded; overflow is only printed.	Return 1/0 status.
3	main() prints 'Popped: -1' after underflow	Output is misleading: it reports an item that never existed.	Print 'Popped' only if pop() succeeded.
4	Global variables	Only one stack can exist; any code can change top.	Wrap stack, top in a struct and pass a pointer to functions.
5	Fixed capacity (SIZE = 5)	Cannot grow.	Use malloc/realloc (dynamic array) or a linked list.
6	Trailing space in display()	Cosmetic; every line ends with a space.	Print separator only between items.
7	pop() / display() declared with empty ()	In C this means 'unspecified arguments' (not void).	Declare as pop(void), display(void).
8	No peek() / isEmpty() / isFull()	Callers must duplicate the top tests.	Add helper functions.

Output of the corrected program (stack_fixed.c), where a stored -1 is correctly popped and the real underflow prints no fake item:

```
Pushed: -1
Popped: -1
Stack Underflow
Stack is Empty
```

8. Execution and Comparison

Compiled with `gcc -Wall -Wextra -o main stack.c` (no warnings) and run. Actual output:

```
Pushed: 10
Pushed: 20
Pushed: 30
Stack: 10 20 30
Popped: 30
Pushed: 40
Pushed: 50
Pushed: 60
Stack Overflow
Stack: 10 20 40 50 60
Popped: 60
Popped: 50
Popped: 40
Popped: 20
Popped: 10
Stack is Empty
Stack Underflow
Popped: -1
```

Comparison: the actual output is identical to the predicted output on all 18 lines (checked with diff). Final state top = -1 confirmed. The manual trace had no mistakes.

The modified program (modified.c) was also run, and its actual output matched its prediction (see section 9, row M).

9. Additional Test Cases

Expected results were written by hand before running; actual results are from run_tests.sh. Each test reuses the original push/pop/display with a different main().

ID	Test	Operations	Expected	Actual	Result
T1	Pop and display on an empty stack	pop(); print; display();	Stack Underflow | Popped: -1 | Stack is Empty	Stack Underflow | Popped: -1 | Stack is Empty	PASS
T2	Fill stack to capacity and push a 6th item	push(1..6); display();	Pushed: 1 | Pushed: 2 | Pushed: 3 | Pushed: 4 | Pushed: 5 | Stack Overflow | Stack: 1 2 3 4 5	Pushed: 1 | Pushed: 2 | Pushed: 3 | Pushed: 4 | Pushed: 5 | Stack Overflow | Stack: 1 2 3 4 5	PASS
T3	Push the value -1, pop it, pop again (sentinel clash)	push(-1); pop(); pop();	Pushed: -1 | Popped: -1 | Stack Underflow | Popped: -1	Pushed: -1 | Popped: -1 | Stack Underflow | Popped: -1	PASS
T4	Fill, pop one, push again (LIFO + reuse of slot)	push(1..5); pop(); push(9); display();	Pushed: 1 | Pushed: 2 | Pushed: 3 | Pushed: 4 | Pushed: 5 | Popped: 5 | Pushed: 9 | Stack: 1 2 3 4 9	Pushed: 1 | Pushed: 2 | Pushed: 3 | Pushed: 4 | Pushed: 5 | Popped: 5 | Pushed: 9 | Stack: 1 2 3 4 9	PASS
M	Modified program (Q6)	modified.c	Pushed: 15 | Pushed: 25 | Pushed: 35 | Stack: 15 25 35 | Popped: 35 | Pushed: 40 | Pushed: 50 | Pushed: 60 | Popped: 60 | Stack: 15 25 40 50 | Popped: 50 | Popped: 40 | Popped: 25 | Popped: 15 | Stack Underflow | Popped: -1 | Stack is Empty | Stack Underflow | Popped: -1	Pushed: 15 | Pushed: 25 | Pushed: 35 | Stack: 15 25 35 | Popped: 35 | Pushed: 40 | Pushed: 50 | Pushed: 60 | Popped: 60 | Stack: 15 25 40 50 | Popped: 50 | Popped: 40 | Popped: 25 | Popped: 15 | Stack Underflow | Popped: -1 | Stack is Empty | Stack Underflow | Popped: -1	PASS

T3 passes, but it exposes limitation 1: the first 'Popped: -1' is real data, while the second line pair is an error; the printed output looks the same for both.

10. Conclusion

The program implements a bounded LIFO stack with an array and a single index, top. Every push and pop is O(1); display is O(n). The manual trace predicted all 18 output lines correctly, including the overflow on push(70) and the underflow on the final pop(). The tests and the modified program confirm that top guards every access, and that the remaining weaknesses are design issues (the -1 sentinel, void push, global state, fixed size) rather than bugs in the LIFO logic. A status-returning pop with an output pointer removes the ambiguity.

Viva preparation

Question	Answer
What is a stack?	A LIFO structure: last item in is the first out; access only at the top.
Why top = -1 initially?	Because index 0 is the first valid slot; -1 means 'no element'. First push does top++ to 0.
Overflow condition?	top == SIZE - 1.
Underflow condition?	top == -1.
Does pop erase the value?	No, it only decrements top; the old value stays in memory but is inaccessible.
Time complexity?	push, pop O(1); display O(n).
Why does display not print after the last pop?	top = -1, so the 'Stack is Empty' branch runs and the loop is skipped.
What is the weakness of returning -1 from pop?	It cannot be told apart from a stored -1; use a status code plus pointer.
Where are stacks used?	Function calls, undo, expression evaluation, bracket matching, DFS, backtracking.

Appendix A - Source Code (stack.c)

```
#include <stdio.h>
#define SIZE 5

int stack[SIZE];
int top = -1;

void push(int item)
{
    if (top == SIZE - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        top++;
        stack[top] = item;
        printf("Pushed: %d\n", item);
    }
}

int pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }
    else
    {
        int item = stack[top];
        top--;
        return item;
    }
}

void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Stack: ");
        for (i = 0; i <= top; i++)
        {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main()
{
    int item;

    push(10);
    push(20);
    push(30);

    display();

    item = pop();
    printf("Popped: %d\n", item);

    push(40);
    push(50);
    push(60);
    push(70);

    display();

    item = pop();
    printf("Popped: %d\n", item);
```

```
    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    display();

    item = pop();
    printf("Popped: %d\n", item);

    return 0;
}
```

Appendix B - Modified Program (modified.c)

```
#include <stdio.h>
#define SIZE 5

int stack[SIZE];
int top = -1;

void push(int item)
{
    if (top == SIZE - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        top++;
        stack[top] = item;
        printf("Pushed: %d\n", item);
    }
}

int pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }
    else
    {
        int item = stack[top];
        top--;
        return item;
    }
}

void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is Empty\n");
    }
    else
    {
        printf("Stack: ");
        for (i = 0; i <= top; i++)
        {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main()
{
    int item;

    push(15);
    push(25);
    push(35);

    display();

    item = pop();
    printf("Popped: %d\n", item);

    push(40);
    push(50);
    push(60);
    item = pop();
    printf("Popped: %d\n", item);

    display();

    item = pop();
```

```
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    item = pop();
    printf("Popped: %d\n", item);

    display();

    item = pop();
    printf("Popped: %d\n", item);

    return 0;
}
```

Appendix C - Test Programs (main() bodies, tests/)

T1

```
int main(){int item; item=pop(); printf("Popped: %d\n",item); display(); return 0;}
```

T2

```
int main(){int i; for(i=1;i<=6;i++) push(i); display(); return 0;}
```

T3

```
int main(){int item; push(-1); item=pop(); printf("Popped: %d\n",item); item=pop(); printf("Popped: %d\n",item); retu
```

T4

```
int main(){int i,item; for(i=1;i<=5;i++) push(i); item=pop(); printf("Popped: %d\n",item); push(9); display(); return
```

Appendix D - Improved Version (stack_fixed.c)

```
#include <stdio.h>
#define SIZE 5

/* Improved version: push/pop return a status (1 = success, 0 = failure),
   and pop hands the value back through a pointer, so a stored -1 can no
   longer be confused with the "underflow" error value. */
int stack[SIZE];
int top = -1;

int push(int item)
{
    if (top == SIZE - 1) { printf("Stack Overflow\n"); return 0; }
    stack[++top] = item;
    printf("Pushed: %d\n", item);
    return 1;
}

int pop(int *item)
{
    if (top == -1) { printf("Stack Underflow\n"); return 0; }
    *item = stack[top--];
    return 1;
}

void display(void)
{
    int i;
    if (top == -1) { printf("Stack is Empty\n"); return; }
    printf("Stack:");
    for (i = 0; i <= top; i++) printf(" %d", stack[i]);
    printf("\n");
}

int main(void)
{
    int item;
    push(-1);
    if (pop(&item)) printf("Popped: %d\n", item);   /* prints Popped: -1 (a real value) */
    if (pop(&item)) printf("Popped: %d\n", item);   /* prints Stack Underflow only      */
    display();
    return 0;
}
```

Appendix E - Test Script Result

```
PASS  main
PASS  modified
PASS  t1
PASS  t2
PASS  t3
PASS  t4
Passed: 6  Failed: 0
```